

Link GitHub: https://jonyad.github.io/Pagina_Web_Gaming/

Con este documento quiero explicar todas las funcionalidades de JavaScript que he ido añadiendo a la página. Para cada una indico en qué fichero está, dónde se puede ver y cómo funciona por dentro.

1. RELOJ EN TIEMPO REAL

Fichero: js/reloj.js

Visible en: todas las páginas, esquina superior izquierda del header.

Al abrir cualquier página se ve un reloj digital que muestra la hora actual y se actualiza cada segundo.

Para verlo: abrir cualquier página y observar la esquina superior izquierda del header.

Cómo funciona:

- Se usa el objeto `Date()` de JavaScript para obtener la hora actual del sistema.
 - Con `getHours()`, `getMinutes()` y `getSeconds()` se obtiene cada parte.
 - Si el número es menor que 10 se le añade un cero delante con un `if`, para que se muestre 09 en vez de 9.
 - Con `setInterval` la función se repite cada 1000 milisegundos (1 segundo).
-

2. SISTEMA DE LOGIN CON `localStorage`

Fichero: js/login.js

Visible en: panel lateral derecho (aside) de todas las páginas.

El aside muestra un formulario con usuario y contraseña. Si los datos son correctos, desaparece el formulario y aparece el perfil completo con nombre, avatar y estadísticas del jugador. Si son incorrectos, salta un alert avisando del error.

Credenciales para probarlo: Usuario: jonathan Contraseña: esto es una contraseña

Las credenciales se han simplificado para facilitar la evaluación. En un entorno real, la contraseña seguiría criterios de seguridad más estrictos.

Para verlo: ir al aside de cualquier página, introducir las credenciales y pulsar Entrar.

Cómo funciona:

- Se obtienen los valores de los inputs con `getElementById` y `.value`.
- Se comparan con el usuario y contraseña correctos usando un `if`.
- Si es correcto, se guarda en `localStorage` con `setItem` para que la sesión se mantenga aunque se cambie de página o se recargue.

- Se oculta el formulario (`display: none`) y se muestra el perfil (`display: block`) manipulando el estilo desde JavaScript.
 - Al hacer login también se cambia la imagen del avatar usando `getAttribute` para leer la ruta actual y reemplazando el nombre del archivo por el del avatar del perfil.
 - Al cerrar sesión se borra la entrada de `localStorage` con `removeItem` y se limpian los campos del formulario.
 - Con `window.addEventListener('load')` se comprueba al cargar si ya había una sesión guardada, para no tener que volver a iniciarla cada vez que se cambia de página.
-

3. FEED DE COMUNIDAD EN LA BARRA DE NAVEGACIÓN

Fichero: `js/comunidad.js`

Visible en: parte inferior del menú lateral, en todas las páginas.

Cada 3 segundos aparece automáticamente un mensaje nuevo de un usuario de la comunidad. Se van acumulando hasta un máximo de 3 visibles, y el más antiguo desaparece cuando llega uno nuevo.

Para verlo: esperar unos segundos en cualquier página y observar la parte de abajo del menú lateral izquierdo.

Cómo funciona:

- Los mensajes están guardados en un array de objetos, cada uno con un nombre de usuario y un mensaje.
 - Con `setInterval` se llama a la función cada 3000 ms.
 - Dentro de la función se crea un elemento `<p>` nuevo con `createElement`, se le añade la clase correspondiente con `classList.add`, y se escribe el contenido con `innerHTML` incluyendo el nombre del usuario en un `span`.
 - Se añade a la página con `appendChild`.
 - Se cuentan los mensajes visibles con `querySelectorAll`. Si hay más de 3, se elimina el primero con `.remove()`.
 - Un contador va rotando por el array para que los mensajes salgan en orden y vuelvan a empezar al llegar al final.
-

4. GALERÍA INTERACTIVA DE IMÁGENES

Fichero: `js/galeria.js`

Visible en: página Galería (menú → Galería).

Hay cuatro botones, uno por juego. Al pulsar uno se muestran las imágenes de ese juego. Se puede navegar manualmente con los botones Anterior y Siguiente, o activar el pase automático.

Para verlo: ir a Galería, pulsar cualquier botón de juego y navegar con los botones. Probar también el botón "Pasar Automático".

Cómo funciona:

- Las imágenes están organizadas en un objeto que contiene arrays, uno por juego. Cada elemento tiene la URL, el título y la descripción de la imagen.
 - Al pulsar un botón de juego, se recorren todos los botones con `querySelectorAll` y un bucle `for` para quitarles la clase "activo" con `classList.remove`, y luego se la añade solo al pulsado con `classList.add`.
 - Para navegar, se incrementa o decrementa un índice y se actualiza la imagen, el título, la descripción y el contador en el DOM con `textContent` y asignando el `src` de la imagen.
 - El botón Anterior usa un `if/else`: si el índice es 0 salta al último, si no resta 1. El Siguiente suma 1 y vuelve a 0 cuando llega al final.
 - El modo automático usa `setInterval` para avanzar solo cada 3 segundos, y `clearInterval` para detenerlo. Se controla con una variable que guarda el intervalo activo.
-

5. ANIMACIÓN DE BARRAS EN RANKINGS

Fichero: `js/rankings.js`

Visible en: página Rankings, sección del gráfico de barras.

Al cargar la página Rankings, las barras del gráfico crecen desde 0% hasta su valor real de forma progresiva, en vez de aparecer estáticas.

Para verlo: ir a Rankings y observar cómo crecen las barras al cargar. También se puede recargar la página para verlo de nuevo.

Cómo funciona:

- Se obtienen todas las barras con `querySelectorAll`.
 - Se recorren con un bucle `for`. Para cada barra se lee su anchura final del HTML con `parseInt`, se pone a 0%, y se crea un `setInterval` que va aumentando el valor de 1 en 1 hasta llegar al porcentaje correcto, momento en que se detiene con `clearInterval`.
-

6. PODIO TOP 3

Fichero: `js/rankings.js`

Visible en: página Rankings, justo debajo del gráfico.

Se muestra un podio visual con los 3 juegos más valorados, con su icono y puntuación. El primero va en el centro (oro), el segundo a la izquierda (plata) y el tercero a la derecha (bronce).

Para verlo: ir a Rankings y ver la sección del podio.

Cómo funciona:

- Los datos del ranking están en un array de objetos con el nombre del juego, el porcentaje y la ruta del icono.

- Para ordenarlos de mayor a menor sin usar métodos automáticos, se utilizan dos bucles for anidados: el exterior repite 3 veces (una por puesto del podio) y el interior recorre el array buscando el mayor porcentaje restante. Una vez encontrado, se guarda en el array del podio y se elimina del original para no repetirlo.
 - Con getElementById se actualizan la imagen, el nombre y el porcentaje de cada posición del podio en el HTML.
-

7. VOTAR JUEGO FAVORITO CON GUARDADO

Fichero: js/rankings.js

Visible en: página Rankings, tarjeta "Vota tu juego favorito".

Al seleccionar un juego del desplegable y pulsar Votar, aparece un mensaje de confirmación. El voto se guarda en localStorage, por lo que al volver a la página el voto anterior ya aparece seleccionado.

Para verlo: ir a Rankings, seleccionar un juego, pulsar Votar, cambiar de página y volver a Rankings.

Cómo funciona:

- Se lee el valor del select con getElementById y .value.
 - Si está vacío se avisa con alert.
 - Si tiene valor, se guarda en localStorage con setItem.
 - Se muestra el mensaje de confirmación cambiando el innerHTML del div de resultado, incluyendo el nombre del juego en negrita.
 - Al cargar la página, con window.addEventListener('load') se comprueba si hay un voto guardado en localStorage y, si lo hay, se selecciona automáticamente en el desplegable.
-

8. TRÁILER DE YOUTUBE AL SELECCIONAR JUEGO

Fichero: js/rankings.js

Visible en: página Rankings, dentro de la tarjeta de votos.

Al seleccionar un juego del desplegable aparece a la derecha el tráiler oficial en un vídeo de YouTube. Cambia según el juego que se elija.

Para verlo: ir a Rankings y seleccionar distintos juegos del desplegable. El tráiler aparece a la derecha del formulario.

Nota: para que el vídeo se muestre correctamente, la página debe estar en un servidor (GitHub Pages). Desde archivo local no funciona, por eso está alojado en GitHub.

Cómo funciona:

- Las URLs de los tráilers están guardadas en un objeto JavaScript, con una clave por juego.

- Al seleccionar un juego se asigna la URL correspondiente al src del iframe de YouTube, añadiendo ?rel=0 al final para que no muestre vídeos relacionados al terminar.
 - El div del tráiler empieza oculto (display: none) y se muestra solo cuando hay un juego seleccionado.
-

9. COMENTARIOS CON GUARDADO Y LISTADO

Fichero: js/rankings.js

Visible en: página Rankings, tarjeta "Déjanos tu comentario".

Se puede escribir un comentario y enviarlo. Queda guardado y aparece listado debajo. Los comentarios persisten aunque se recargue la página o se cierre el navegador.

Para verlo: ir a Rankings, escribir algo en el textarea y pulsar Enviar. Recargar la página y comprobar que siguen ahí.

Cómo funciona:

- Los comentarios se guardan en localStorage como un array usando JSON.stringify para convertirlo a texto, ya que localStorage solo guarda texto.
 - Al recuperarlos se usa JSON.parse para volver a convertirlos en array de JavaScript.
 - Para mostrarlos se recorre el array al revés (del último al primero) para que el más reciente aparezca arriba. Por cada comentario se crea un elemento <p> con createElement, se le añade su clase y texto, y se inserta en la página con appendChild.
 - Antes de enviar se comprueba que el textarea no esté vacío con un if, avisando con alert si lo está.
-

10. CONTADORES ANIMADOS EN EL INICIO

Fichero: js/contador.js

Visible en: página de inicio, barra de estadísticas debajo de las tarjetas.

Hay cuatro contadores (juegos analizados, partidas registradas, horas de contenido y usuarios activos) que arrancan en 0 y suben hasta su valor real al cargar la página.

Para verlo: ir a Inicio y observar los contadores de la parte inferior. Recargar para verlos de nuevo.

Cómo funciona:

- Los datos de cada contador están en un array de objetos, con el id del elemento HTML, el valor objetivo y un sufijo opcional (como la h de horas).
- Con window.addEventListener('load') se espera a que la página cargue antes de arrancar.
- Se recorre el array con un bucle for. Para cada contador se crea un setInterval que suma 5 al valor actual cada 25 ms. Cuando llega al objetivo se detiene con clearInterval y se fija el valor exacto para que no se pase.

- Se actualiza el texto del elemento con `textContent` en cada paso, añadiendo el sufijo si lo tiene.
-

11. BOTÓN VOLVER ARRIBA

Fichero: `js/extras.js`

Visible en: esquina inferior derecha de todas las páginas.

Aparece un botón flotante con una flecha cuando el usuario baja más de 200 píxeles en la página. Al pulsarlo vuelve arriba del todo.

Para verlo: ir a cualquier página con bastante contenido, como Rankings o Ejercicios, y hacer scroll hacia abajo.

Cómo funciona:

- Con `window.addEventListener('scroll')` se detecta cada vez que el usuario desplaza la página.
 - Se comprueba `window.scrollY`, que indica cuántos píxeles se ha bajado. Si supera 200 se muestra el botón, si no se oculta, cambiando el display con JavaScript.
 - Al pulsar el botón se llama a `window.scrollTo(0, 0)`, que lleva la página al principio.
-

12. CONTADOR DE VISITAS

Fichero: `js/extras.js`

Visible en: pie de página (footer) de todas las páginas.

Cada vez que se carga una página se suma 1 a un contador que se muestra en el footer. El número se mantiene entre sesiones.

Para verlo: observar el footer de cualquier página y navegar entre páginas para ver cómo sube el número.

Cómo funciona:

- Al cargar la página, con `window.addEventListener('load')`, se lee el valor guardado en `localStorage`.
- Si no existe aún (primera visita) se pone a 1. Si ya existe se convierte a número con `parseInt` y se le suma 1.
- Se guarda el nuevo valor con `setItem` y se actualiza el texto del span del footer con `textContent`.

13. SLIDESHOW DE JUEGO DESTACADO EN EL ASIDE

Fichero: js/extras.js

Visible en: parte inferior del panel lateral derecho, en todas las páginas.

Debajo del perfil hay un pequeño widget que va rotando los cuatro juegos del portal cada 3 segundos, con un efecto de fundido al cambiar.

Para verlo: mirar la parte inferior del aside en cualquier página y esperar unos segundos.

Cómo funciona:

- Los juegos están en un array de objetos con nombre, ruta de imagen y descripción.
- Para que las rutas de las imágenes funcionen tanto desde la página principal como desde las páginas de la carpeta htmls, se comprueba con un if si la URL actual contiene '/htmls/' y se ajusta la ruta en consecuencia.
- Al cambiar de juego, primero se pone la opacidad de la imagen a 0. Luego, con setTimeout se esperan 400 ms y se cambia la imagen y el texto. Al terminar el setTimeout se vuelve a poner la opacidad a 1 y aparece la nueva imagen con un efecto de fundido suave.
- Con setInterval se repite el ciclo cada 3 segundos. El uso combinado de setTimeout dentro de setInterval fue algo que investigué para conseguir el efecto de transición, ya que cambiando la imagen directamente el cambio quedaba brusco.